

The program should run the same on Linux , windows and mac. Give a description of what to install, troubleshoot, or change for this program to work on all 3 operating systems. If you need to rewrite the code, do so, and show all code.

Thought for 7s

The same source can run on Windows, macOS, and Linux. SDL2 (via Pygame 2) is the common audio backend; the differences are **how you install Pygame**, **OS permissions**, and **which sound server SDL talks to**. The rewrite below adds a **Default device** entry (`devicename=None`), decodes names that come back as `bytes` , allows format/channel fallback, and surfaces open errors in the window.

Install (all three OSes)

Use **Python 3.10+** and **Pygame 2.5+** (2.x is required for `pygame._sdl2.AudioDevice`).

```
bash

python3 -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate

pip install -U pip
pip install "pygame>=2.5"
```

`pygame-ce` also works if you prefer it:

```
bash

pip install pygame-ce
```

Do **not** mix `pygame` and `pygame-ce` in the same venv.

Save the program as `audio_io_test.py` and run:

```
bash

python audio_io_test.py
```

No Tk, NumPy, or PortAudio required.

Platform notes

Windows

- Use the official python.org or Store Python. WASAPI/DirectSound is what SDL uses.
- Grant **microphone** access if Windows prompts (Settings → Privacy → Microphone). Allow Python / Terminal / your IDE.

- Exclusive-mode devices (some USB interfaces, “Exclusive Mode” in Sound control panel) can refuse a second open. Uncheck exclusive mode, or pick another device.
- Bluetooth headsets often expose separate “Hands-Free” (narrowband mic+spk) and “Stereo” devices. Use Stereo for the output tone; Hands-Free for the mic if that is all you have.
- If the list is empty: `set SDL_AUDIODRIVER=wasapi` then rerun. Fallback: `directsound`.

macOS

- First capture open triggers **Microphone** permission for Terminal, iTerm, Python, or your IDE.
System Settings → Privacy & Security → Microphone → enable that app.
If you clicked Don’t Allow, toggle it on and restart the program.
- Built-in mic + speakers usually work. Aggregate devices and some Bluetooth profiles can fail format negotiation; the code retries mono / format change.
- Fonts: Menlo / Helvetica are used if Segoe UI is missing.
- Apple Silicon: a native `arm64` Python + Pygame wheel is required (not an `x86_64` venv under Rosetta unless Pygame is also `x86_64`).

Linux

SDL talks to **PipeWire**, **PulseAudio**, or raw **ALSA**.

Typical packages:

```
bash

# Debian/Ubuntu
sudo apt install python3-venv libsndl2-2.0-0 libsndl2-mixer-2.0-0 pulseaudio-utils
# Fedora
sudo dnf install python3-pip SDL2 SDL2_mixer
# Arch
sudo pacman -S python-pip sdl2 sdl2_mixer pipewire pipewire-pulse
```

- Confirm a server is running: `pactl info` or `pw-cli info 0`.
- Microphone access: desktop portals / Flatpak sandboxes can block capture. Run the script **outside** Flatpak if the mic list is empty.
- WSL2 usually has **no** real sound card unless WSLg/Pulse is set up; ALSA “card 0” errors mean that.
- If devices are missing:

```
bash

export SDL_AUDIODRIVER=pulse
# or
export SDL_AUDIODRIVER=pipewire
# last resort
export SDL_AUDIODRIVER=alsa
```

`pipewire-pulse` should be running on modern distros so `pulse` still works.

Troubleshooting

Symptom	What to try
<code>ModuleNotFoundError: pygame</code>	Activate the venv; <code>pip install pygame</code>
<code>cannot import AudioDevice</code>	Pygame 1.x — upgrade to 2.5+
Empty input/output lists	Mixer must init first (the script does this). Set <code>SDL_AUDIODRIVER</code> . On Linux start PipeWire/Pulse. On Mac grant mic permission and reboot the app.
"Failed to open ..." in the status bar	Another app has exclusive access; try Default . On Windows disable exclusive mode. On Linux check <code>pavucontrol</code> that the app isn't muted.
Tone plays, scope flat on input	Mic muted in OS mixer; wrong device (monitor vs mic); Mac privacy off.
Distorted / wrong speed	Device forced a different rate. <code>AUDIO_ALLOW_FORMAT_CHANGE</code> is on; if it is still wrong, pick Default.
Crash in the audio callback	Keep callback work tiny (this code only copies floats). Don't call Pygame display APIs from the callback.
Wayland / HiDPI odd window size	Harmless; the UI is laid out from <code>get_size()</code> each frame.

Print the driver SDL actually chose (optional, in a shell):

```
bash
python -c "import os; print('SDL_AUDIODRIVER=', os.environ.get('SDL_AUDIODRIVER'))"
```

Full program

```
python
#!/usr/bin/env python3
"""
Cross-platform audio I/O test (Windows / macOS / Linux).

Pygame 2 + SDL2 only. No Tk.

Layout left -> right:
[outputs] [input oscilloscope] [output oscilloscope] [inputs]
"""

from __future__ import annotations

import math
import sys
import threading
from array import array
```

```

import pygame as pg

try:
    from pygame._sdl2 import (
        AUDIO_ALLOW_FORMAT_CHANGE,
        AUDIO_F32,
        AudioDevice,
        get_audio_device_names,
    )
except ImportError as exc:
    sys.exit(
        "This program needs Pygame 2.5+ with pygame._sdl2.AudioDevice.\n"
        "Install: pip install 'pygame>=2.5'\n"
        f"Import error: {exc}"
    )

try:
    from pygame._sdl2 import AUDIO_F32SYS
except ImportError:
    AUDIO_F32SYS = AUDIO_F32

SAMPLE_RATE = 44100
CHANNELS = 1
CHUNK = 512
SCOPE_SAMPLES = 1024
TONE_HZ = 440.0
TONE_AMP = 0.18
DEFAULT_LABEL = "Default device"

BG = (18, 20, 26)
PANEL = (28, 32, 40)
TEXT = (220, 224, 230)
TEXT_DIM = (140, 148, 160)
ACCENT_IN = (80, 200, 160)
ACCENT_OUT = (90, 160, 240)
GRID = (40, 46, 56)
SELECT = (70, 90, 120)
ERR = (230, 120, 110)

def _as_str(name) -> str:
    if isinstance(name, bytes):
        return name.decode("utf-8", errors="replace")
    return str(name)

def list_devices(capture: bool) -> list[str]:
    """Enumerate SDL devices. Mixer must be initialized first."""
    if pg.mixer.get_init() is None:
        try:
            pg.mixer.init(
                frequency=SAMPLE_RATE, size=32, channels=2, buffer=CHUNK
            )
        except pg.error:
            try:
                pg.mixer.init()
            except pg.error:
                return []
    try:
        raw = get_audio_device_names(capture)
    except Exception:
        return []
    names = []
    seen = set()
    for n in raw:
        s = _as_str(n).strip()

```

```

        if s and s not in seen:
            seen.add(s)
            names.append(s)
    return names

```

```

class RingBuffer:
    def __init__(self, n: int):
        self.n = n
        self.data = [0.0] * n
        self.i = 0
        self.lock = threading.Lock()

    def push(self, samples):
        with self.lock:
            for s in samples:
                self.data[self.i] = float(s)
                self.i = (self.i + 1) % self.n

    def snapshot(self) -> list[float]:
        with self.lock:
            i = self.i
            return self.data[i:] + self.data[:i]

```

```

class AudioIO:
    def __init__(self):
        self.in_buf = RingBuffer(SCOPE_SAMPLES)
        self.out_buf = RingBuffer(SCOPE_SAMPLES)
        self.phase = 0.0
        self.phase_step = 2.0 * math.pi * TONE_HZ / SAMPLE_RATE
        self.capture = None
        self.playback = None
        self.in_name = None
        self.out_name = None
        self.last_error = ""

    def _on_capture(self, _dev, memview):
        raw = bytes(memview)
        samples = array("f")
        samples.frombytes(raw)
        if len(samples) >= 2:
            # Downmix whatever channel count SDL actually delivered.
            # Heuristic: treat as interleaved stereo if even and loudness similar.
            n = len(samples)
            # Always take first channel of each frame if stereo-sized chunks.
            if n % 2 == 0 and n > 2:
                left = samples[0::2]
                self.in_buf.push(left)
            return
        self.in_buf.push(samples)

    def _on_playback(self, _dev, memview):
        n_bytes = len(memview)
        n_floats = max(1, n_bytes // 4)
        out = array("f")
        ph = self.phase
        step = self.phase_step
        for _ in range(n_floats):
            v = math.sin(ph) * TONE_AMP
            ph += step
            if ph > 2.0 * math.pi:
                ph -= 2.0 * math.pi
            out.append(v)
        self.phase = ph
        raw = out.tobytes()

```

```

# memoryview from SDL may be read-only on some builds; copy via bytes assign
try:
    memview[: len(raw)] = raw
except TypeError:
    pass
self.out_buf.push(out)

def close_input(self):
    if self.capture is not None:
        try:
            self.capture.pause(1)
        except Exception:
            pass
        self.capture = None
    self.in_name = None

def close_output(self):
    if self.playback is not None:
        try:
            self.playback.pause(1)
        except Exception:
            pass
        self.playback = None
    self.out_name = None

def _open(self, name, iscapture):
    """name is None => SDL default device (most portable)."""
    last_err = None
    for ch in (1, 2):
        try:
            dev = AudioDevice(
                devicename=name,
                iscapture=iscapture,
                frequency=SAMPLE_RATE,
                audioformat=AUDIO_F32SYS,
                numchannels=ch,
                chunksize=CHUNK,
                allowed_changes=AUDIO_ALLOW_FORMAT_CHANGE,
                callback=self._on_capture if iscapture else self._on_playback,
            )
            dev.pause(0)
            return dev
        except Exception as e:
            last_err = e
    raise last_err if last_err else RuntimeError("open failed")

def open_input(self, name):
    self.close_input()
    label = DEFAULT_LABEL if name is None else name
    try:
        self.capture = self._open(name, True)
        self.in_name = label
        self.last_error = ""
    except Exception as e:
        self.capture = None
        self.in_name = None
        self.last_error = f"Input '{label}' failed: {e}"
        print(self.last_error, file=sys.stderr)

def open_output(self, name):
    self.close_output()
    label = DEFAULT_LABEL if name is None else name
    try:
        self.playback = self._open(name, False)
        self.out_name = label
        self.last_error = ""

```

```

    except Exception as e:
        self.playback = None
        self.out_name = None
        self.last_error = f"Output '{label}' failed: {e}"
        print(self.last_error, file=sys.stderr)

def close_all(self):
    self.close_input()
    self.close_output()

class DeviceList:
    def __init__(self, title, items, accent):
        self.title = title
        self.items = items
        self.accent = accent
        self.selected = 0
        self.scroll = 0
        self.rect = pg.Rect(0, 0, 0, 0)
        self.row_h = 26

    def set_rect(self, r):
        self.rect = r

    def item_rects(self):
        header = 32
        vis = max(1, (self.rect.height - header) // self.row_h)
        start = self.scroll
        end = min(len(self.items), start + vis)
        rects = []
        for i in range(start, end):
            y = self.rect.y + header + (i - start) * self.row_h
            rects.append(
                (i, pg.Rect(self.rect.x + 4, y, self.rect.width - 8, self.row_h - 2))
            )
        return rects

    def click(self, pos):
        if not self.rect.collidepoint(pos):
            return None
        for i, r in self.item_rects():
            if r.collidepoint(pos):
                self.selected = i
                return i
        return None

    def wheel(self, pos, dy):
        if not self.rect.collidepoint(pos):
            return
        header = 32
        vis = max(1, (self.rect.height - header) // self.row_h)
        max_scroll = max(0, len(self.items) - vis)
        self.scroll = max(0, min(max_scroll, self.scroll - dy))

    def draw(self, surf, font, font_sm):
        pg.draw.rect(surf, PANEL, self.rect, border_radius=6)
        surf.blit(font.render(self.title, True, self.accent), (self.rect.x + 8, self.rect.y + 6))
        for i, r in self.item_rects():
            if i == self.selected:
                pg.draw.rect(surf, SELECT, r, border_radius=4)
                text = self.items[i]
                img = font_sm.render(text, True, TEXT)
                max_w = r.width - 8
                while img.get_width() > max_w and len(text) > 3:
                    text = text[:-2]
                img = font_sm.render(text + "...", True, TEXT)

```

```

        surf.blit(img, (r.x + 4, r.y + (r.height - img.get_height()) // 2))

def draw_scope(surf, rect, samples, color, title, font):
    pg.draw.rect(surf, PANEL, rect, border_radius=6)
    inner = rect.inflate(-16, -48)
    inner.y += 14
    surf.blit(font.render(title, True, color), (rect.x + 10, rect.y + 8))
    pg.draw.rect(surf, (16, 18, 22), inner)
    for g in range(1, 4):
        y = inner.y + inner.height * g / 4
        pg.draw.line(surf, GRID, (inner.x, y), (inner.right, y))
    mid = inner.y + inner.height // 2
    pg.draw.line(surf, GRID, (inner.x, mid), (inner.right, mid))
    if not samples:
        return
    n = len(samples)
    h2 = inner.height / 2
    pts = []
    for i, s in enumerate(samples):
        x = inner.x + int(i * (inner.width - 1) / max(1, n - 1))
        y = mid - int(max(-1.0, min(1.0, s)) * (h2 - 2))
        pts.append((x, y))
    if len(pts) >= 2:
        pg.draw.aalines(surf, color, False, pts)

def layout(w, h):
    pad = 10
    status_h = 26
    list_w = max(180, int(w * 0.20))
    mid = w - 2 * list_w - 4 * pad
    scope_w = max(160, mid // 2)
    y = pad
    hh = h - 2 * pad - status_h
    out_list = pg.Rect(pad, y, list_w, hh)
    in_scope = pg.Rect(out_list.right + pad, y, scope_w, hh)
    out_scope = pg.Rect(in_scope.right + pad, y, scope_w, hh)
    in_list = pg.Rect(w - pad - list_w, y, list_w, hh)
    return out_list, in_scope, out_scope, in_list

def pick_font(size):
    names = (
        "segoeui",
        "segoe ui",
        "helvetica",
        "menlo",
        "dejavusans",
        "liberation sans",
        "nimbus sans",
        "arial",
    )
    return pg.font.SysFont("".join(names), size)

def device_arg(label: str):
    return None if label == DEFAULT_LABEL else label

def main():
    pg.mixer.pre_init(SAMPLE_RATE, 32, 2, CHUNK)
    pg.init()
    try:
        pg.mixer.init(frequency=SAMPLE_RATE, size=32, channels=2, buffer=CHUNK)
    except pg.error as e:

```

```

    print("mixer.init warning:", e, file=sys.stderr)

pg.display.set_caption("Audio I/O test - oscilloscopes")
screen = pg.display.set_mode((1100, 540), pg.RESIZABLE)
clock = pg.time.Clock()
font = pick_font(16)
font_sm = pick_font(13)

outputs = [DEFAULT_LABEL] + list_devices(False)
inputs = [DEFAULT_LABEL] + list_devices(True)

audio = AudioIO()
out_ui = DeviceList("OUTPUTS", outputs, ACCENT_OUT)
in_ui = DeviceList("INPUTS", inputs, ACCENT_IN)

# Default device is the most portable first choice on all three OSes.
audio.open_output(None)
audio.open_input(None)

running = True
while running:
    for ev in pg.event.get():
        if ev.type == pg.QUIT:
            running = False
        elif ev.type == pg.VIDEORESIZE:
            screen = pg.display.set_mode(ev.size, pg.RESIZABLE)
        elif ev.type == pg.MOUSEBUTTONDOWN and ev.button == 1:
            i = out_ui.click(ev.pos)
            if i is not None:
                audio.open_output(device_arg(outputs[i]))
            j = in_ui.click(ev.pos)
            if j is not None:
                audio.open_input(device_arg(inputs[j]))
        elif ev.type == pg.MOUSEWHEEL:
            pos = pg.mouse.get_pos()
            out_ui.wheel(pos, ev.y)
            in_ui.wheel(pos, ev.y)
        elif ev.type == pg.KEYDOWN and ev.key == pg.K_ESCAPE:
            running = False

    w, h = screen.get_size()
    r_out, r_in_sc, r_out_sc, r_in = layout(w, h)
    out_ui.set_rect(r_out)
    in_ui.set_rect(r_in)

    screen.fill(BG)
    out_ui.draw(screen, font, font_sm)
    in_ui.draw(screen, font, font_sm)
    draw_scope(
        screen,
        r_in_sc,
        audio.in_buf.snapshot(),
        ACCENT_IN,
        "INPUT " + (audio.in_name or "(none)"),
        font_sm,
    )
    draw_scope(
        screen,
        r_out_sc,
        audio.out_buf.snapshot(),
        ACCENT_OUT,
        "OUTPUT " + (audio.out_name or "(none)"),
        font_sm,
    )
    status = audio.last_error or (
        f"{sys.platform} | pygame {pg.version.ver} | "

```

```

        f"in: {len(inputs) - 1}  out: {len(outputs) - 1}  |  Esc quit"
    )
    color = ERR if audio.last_error else TEXT_DIM
    screen.blit(font_sm.render(status, True, color), (12, h - 22))
    pg.display.flip()
    clock.tick(60)

audio.close_all()
pg.quit()

if __name__ == "__main__":
    main()

```

What changed for portability: a **Default device** row maps to **devicename=None** (SDL's recommended default on every OS); names are decoded if SDL returns **bytes** ; open retries 1 then 2 channels with **AUDIO_F32SYS** + **AUDIO_ALLOW_FORMAT_CHANGE** ; errors show in the status line instead of dying; fonts are chosen from a cross-OS list. Start there, then click a named device if you need a specific interface.

Explore SDL Audio Driver Configuration

Investigate PyAudio PortAudio Alternatives